

PROJECT DOCUMENTATION REPORT

Smart Internship Skill Intelligence System

1. Title and Project Overview

Project Title: Smart Internship Skill Intelligence System (Full-Stack Web Application)

Student Name: Harsha K

Registration Number: 922525148033

Department: AIML

Project Repository: <https://github.com/harshakolandasamy/Smart-Internship-Skill-Intelligence>

Overview:

The Smart Internship Skill Intelligence System is a complete full-stack CRUD web application developed in accordance with standard operating procedures. The project separates the user interface from the backend server logic and communicates through RESTful APIs. It allows students to manage their profiles, handle multiple technical skills, explore matched internship opportunities, update details, and remove records with immediate persistence in a relational database.

2. Problem Statement

Traditional internship and placement record-keeping is often inefficient, prone to manual entry errors, and lacks fast multi-skill matching capabilities. Monolithic architectures may also combine business logic and user interface into a single layer, making scalability more difficult. This project addresses these issues through a responsive, separated client-server application that provides data access, validation, dynamic skill handling, and automated data persistence.

3. Objectives

- Design and deploy a decoupled full-stack architecture using modern frontend UI components and Python Django REST Framework.
- Provide complete CRUD (Create, Read, Update, Delete) functionality for Student Profiles, Skills, and Internships with dynamic frontend feedback.
- Persist data reliably in an SQLite relational database using Django ORM.
- Implement client-side validation to maintain data integrity.
- Provide an intuitive multi-skill matching and recommendation mechanism.
- Validate REST API endpoints using standard HTTP response testing.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React.js / Vite / Modern UI	Dynamic client-side interface

	Cards	and interactive dashboards
Frontend Styling	Custom CSS / Modern UI Cards	Theme styling, responsive layouts, and card-based data presentation
Backend Framework	Python, Django 6.x / Django REST Framework (DRF)	Server-side routing, serializers, and REST API controllers
Cross-Origin Handler	django-cors-headers	Handles Cross-Origin Resource Sharing (CORS)
Database	SQLite3	Persistent relational data storage via Django ORM
Testing & Tools	Web Browser / REST Client / VS Code	API inspection, runtime execution, and debugging

5. System Architecture

The application follows a decoupled client-server web architecture:

User Browser (Client) ↔ REST API Endpoints (Django) ↔ Django ORM ↔ SQLite Database

- Client Layer: Renders the user interface, validates user inputs, and sends asynchronous HTTP requests (GET, POST, PUT, DELETE) with JSON data.
- REST API Layer: Receives HTTP requests, validates data through serializers, executes business logic, and returns JSON responses with suitable HTTP status codes.
- Database Layer: Django ORM maps model classes to SQLite database tables and stores the data.

6. Database Design and Schema

The primary entities managed by the system are Student, Skill, and Internship.

Entity 1: Student (students_student)

- id (Integer, Primary Key, Auto-Increment): Unique identifier for each student.
- name (Varchar(150), Not Null): Full name of the student.
- email (Varchar(200), Not Null, Unique): Contact email address.
- department (Varchar(100), Not Null): Branch or department name.
- cgpa (Float, Not Null): Academic performance score.

Entity 2: Skill (skills_skill)

- id (Integer, Primary Key, Auto-Increment): Unique identifier.
- student_id (ForeignKey, Not Null): Relational link to the student.
- skill_name (Varchar(255), Not Null): Technical skill such as Python, React, or SQL.

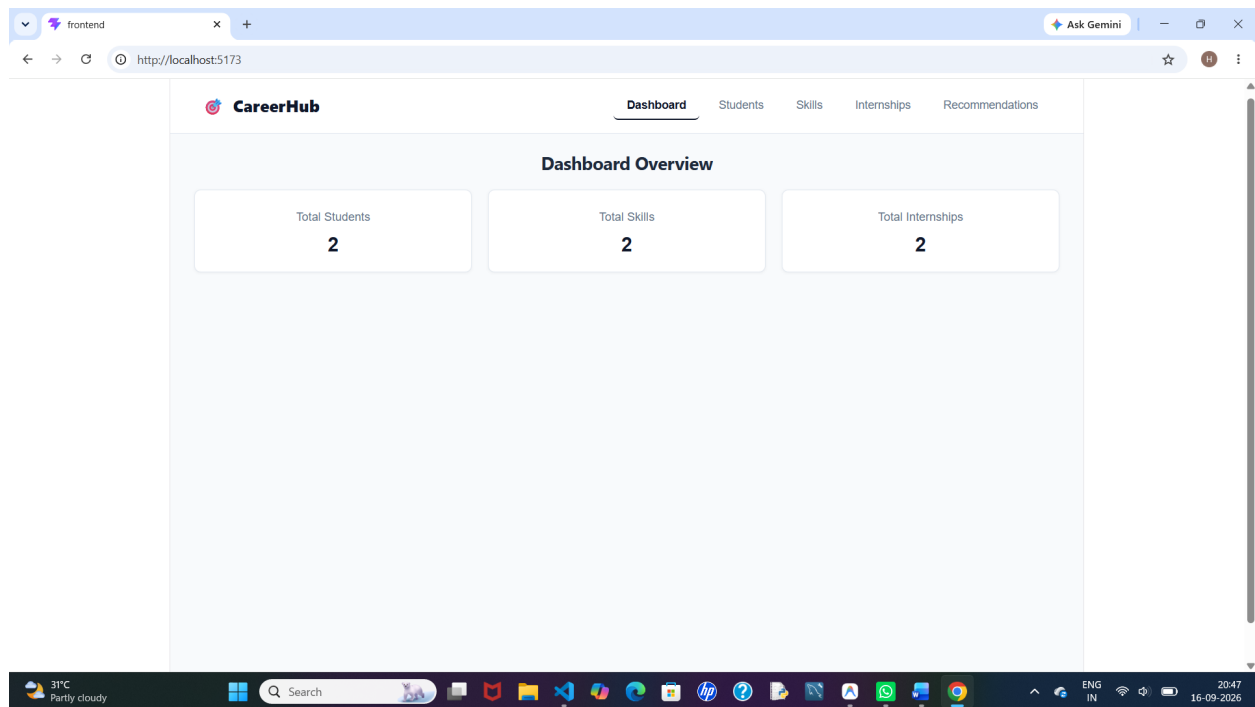
Entity 3: Internship (internships_internship)

- id (Integer, Primary Key, Auto-Increment): Unique identifier.
- company_name (Varchar(150), Not Null): Recruiting company or MNC name.

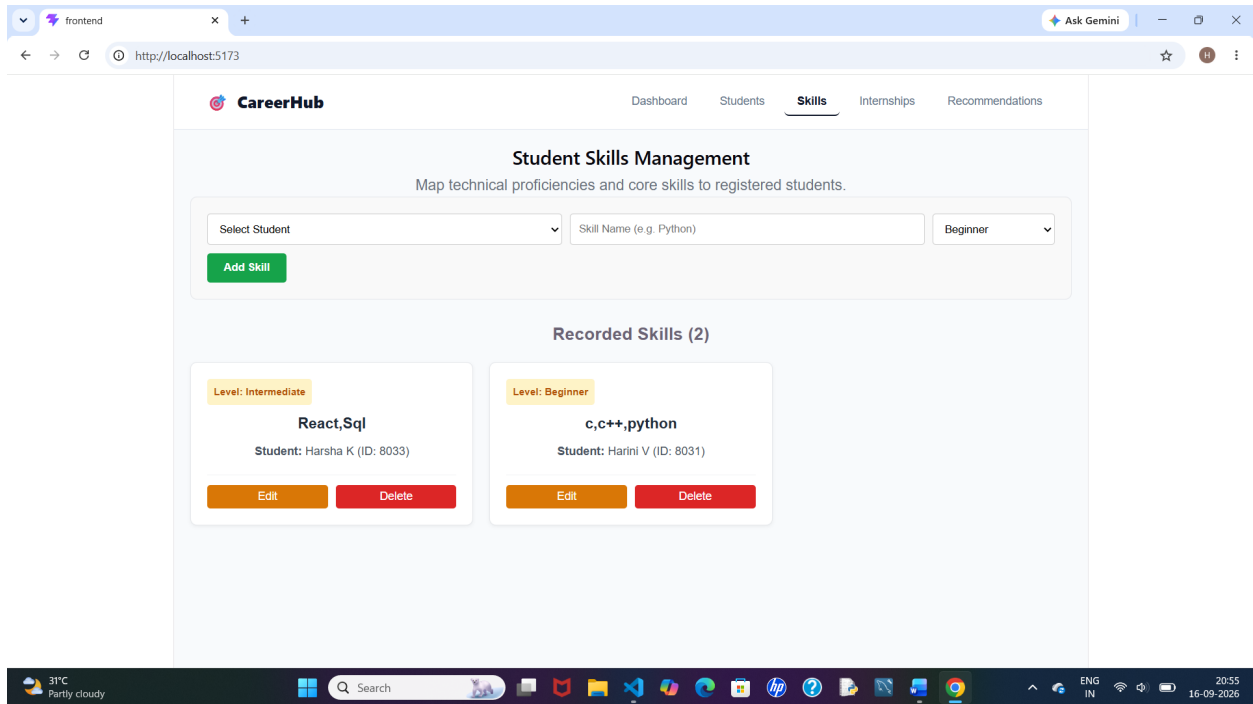
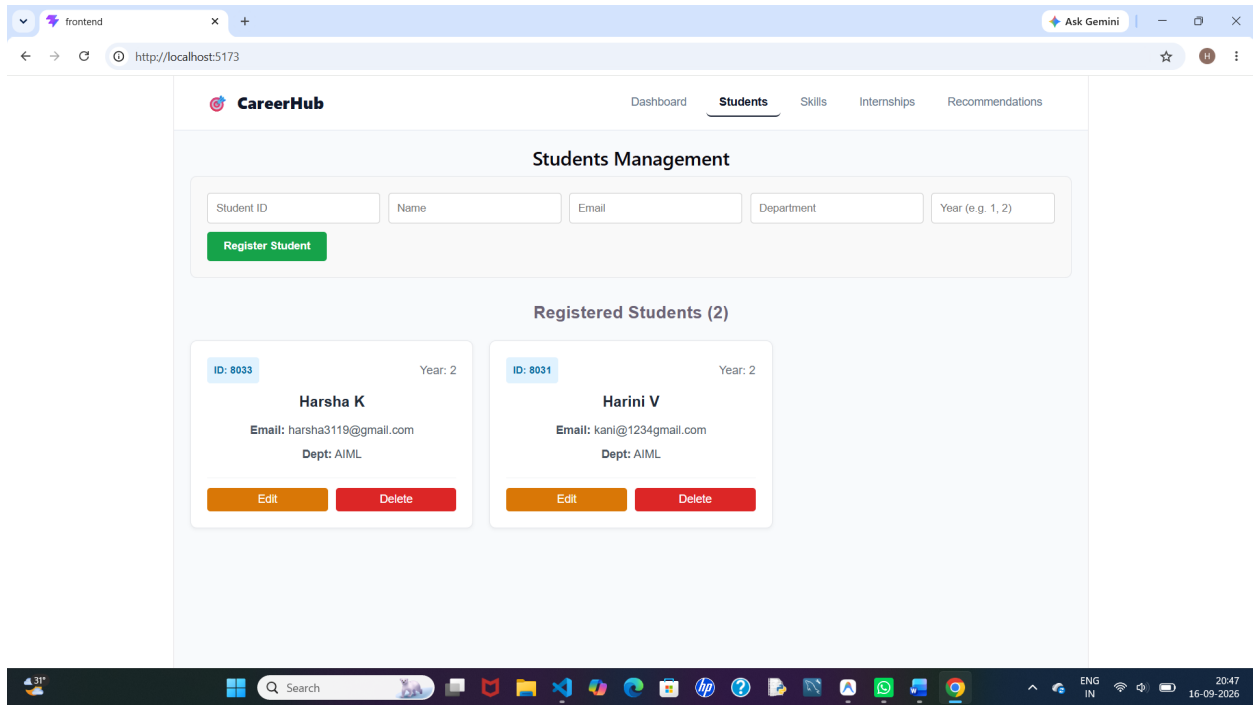
- role (Varchar(150), Not Null): Internship job role.
- required_skills (Text, Not Null): Comma-separated required competencies.

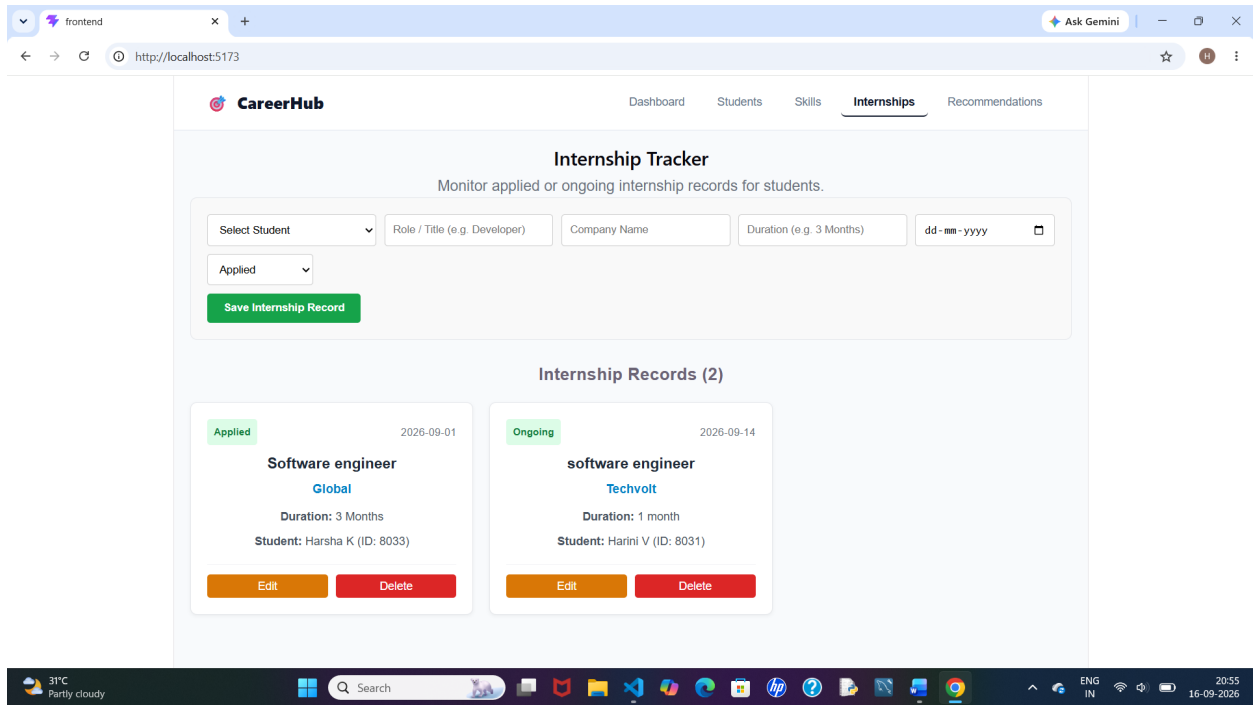
7. User Interface Screenshots

Home Screen (Dashboard and Interactive Cards)

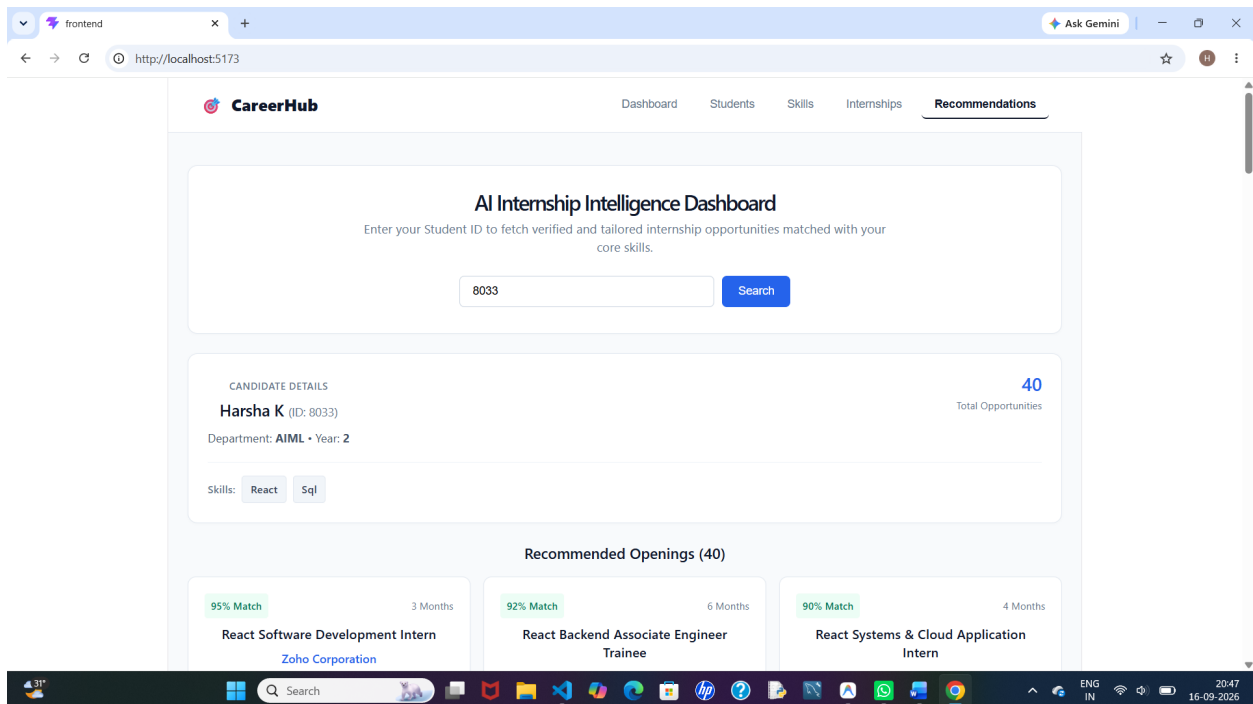


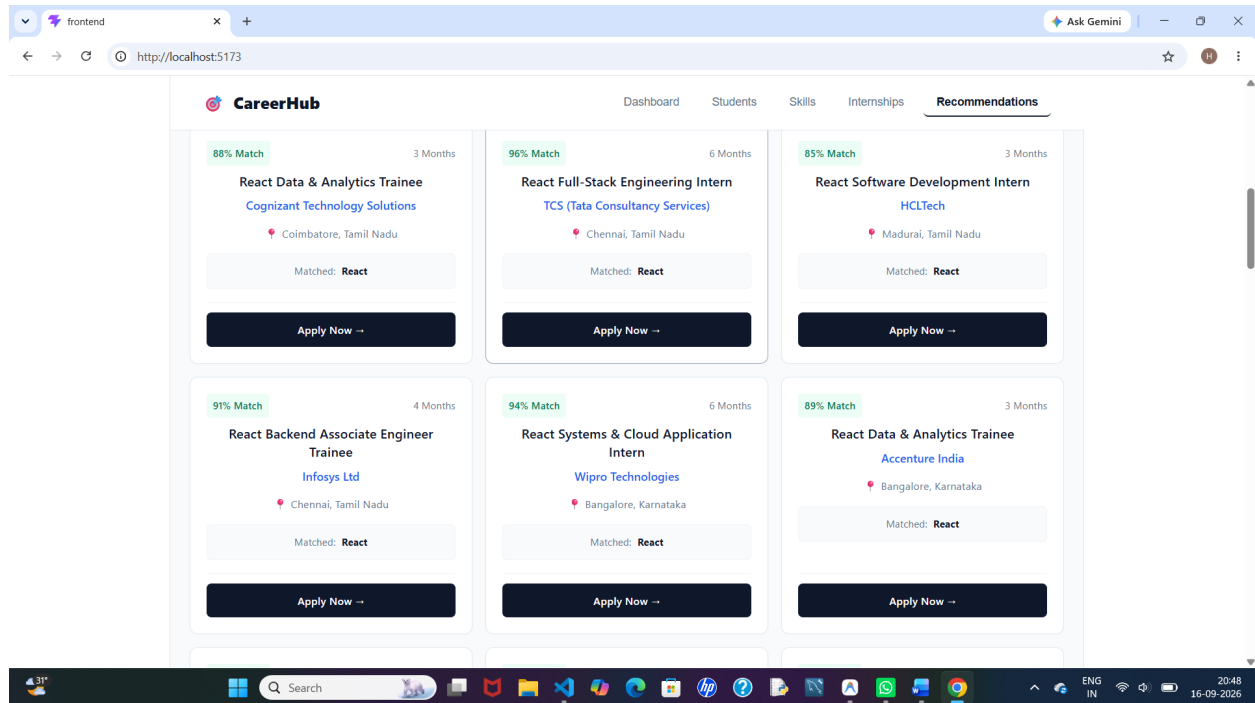
Create Record (Add New Student/Skill/Internship Modal)





Dynamic Recommendation and Search Filtering





8. REST API Endpoint Documentation

Operation	HTTP Method	Endpoint URI	Request Body	Expected Result
Read All	GET	/api/students/	None	Retrieves all student records
Read One	GET	/api/students/{id}/	None	Retrieves a specific student
Create	POST	/api/students/	JSON object	Creates a new record in SQLite DB
Update	PUT	/api/students/{id}/	JSON object	Updates an existing student
Delete	DELETE	/api/students/{id}/	None	Removes the student record
Recommend	GET	/api/recommend-internships/{id}/	None	Returns multi-skill internship matches

9. CRUD Implementation Details

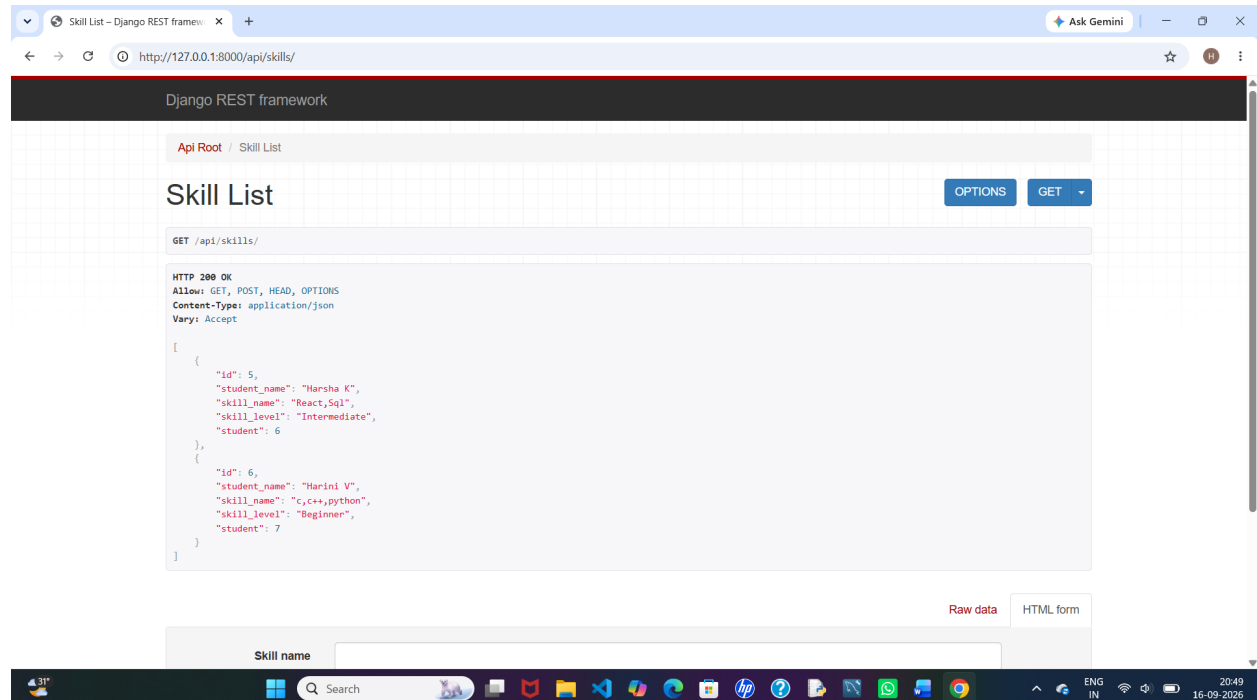
- **Create (POST):** Users enter details in the form. The frontend converts the data into JSON and sends a POST request to the API. The server saves the record and the interface updates.
- **Read (GET):** When the application loads, GET requests retrieve records. The returned JSON data is displayed through dynamic UI cards.

- Update (PUT): Clicking the Edit button loads the selected record into the form. Submitting the form sends a PUT request to update the backend record.
- Delete (DELETE): Clicking Delete asks for confirmation. After confirmation, a DELETE request removes the record from SQLite.

10. Testing Results and Verification

Test Case	Tested Action	Input Data	Expected Output	Status
TC-01	Load Dashboard	Page Load	Fetch all records via GET endpoints	PASSED
TC-02	Add Record	Valid Data	Save to DB and return 201 Created	PASSED
TC-03	Missing Fields	Empty Form	Client-side validation stops submission	PASSED
TC-04	Update Record	Edited Data	Database record updated and 200 OK	PASSED
TC-05	Delete Record	Click Delete and Confirm	Record removed and 204 No Content	PASSED

API Response Verification (Django REST Framework)



11. Installation and Execution Steps

Prerequisites:

- Python 3.10+ installed
- Node.js and npm installed
- Google Chrome or Microsoft Edge browser

Execution Steps:

- Open the backend directory in the terminal: `cd backend`
- Install Python dependencies: `pip install django djangorestframework django-cors-headers`
- Run database migrations: `python manage.py migrate`
- Start the Django development server: `python manage.py runserver`
- Open a new terminal and navigate to the frontend folder: `cd frontend`
- Install frontend packages: `npm install`
- Start Vite: `npm run dev`

12. Challenges and Solutions

CORS Blockage:

- Problem: The browser blocked JavaScript `fetch()` requests because the frontend and backend were running on different origins.
- Solution: Integrated `django-cors-headers` middleware and configured CORS settings in `settings.py`.

Real-Time UI Synchronization:

- Problem: Changes made through Create, Update, or Delete were not immediately visible without refreshing the browser.
- Solution: Used `async/await` fetch handlers and re-fetched the latest data after successful mutation requests.

13. Future Enhancements

- Machine Learning Integration: Implement TF-IDF or Cosine Similarity to calculate more precise job-description matches.
- Automated Resume Parsing: Allow students to upload PDF or Word resumes and automatically extract skills.
- Advanced Authentication and Roles: Integrate JWT-based secure login for students, recruiters, and administrators.

14. Git Repository and Reference Details

- Project Repository: <https://github.com/harshakolandasamy/Smart-Internship-Skill-Intelligence>
- Version Control Tool: Git and GitHub Web
- Primary Branch: `main`

- Commit History: Working source code committed and pushed to the remote repository.
- Compliance: Standard Operating Procedure (SOP) – Complete CRUD-Based Web Application Development Guidelines.